DI5 - Polytech

Ramel JY

TP – Analyse d'images et de vidéos

TP sous Python (Jupyter Notebook ou autres) avec OpenCV principalement.

Ma Python version = 3.6.10 |Anaconda, Inc.| OPENCV Version = 3.4.2

Il est possible d'utiliser Anaconda installé sur les machines des salles TP en creant votre environnement conda dans D:\VM_Utilisateurs

```
conda create - - prefix D:\VM_Utilisateurs\envs/jy python=3.7
conda activate D:\VM_Utilisateurs\envs\jy
conda install -c conda-forge opencv
pip install opencv-contrib-python
conda install -c anaconda jupyter
```

Tous les exercices des TP devront donner lieu à un descriptif/commentaire qui sera intégré dans le compte-rendu des TP à remettre après la fin des TP pour évaluation. Ces rapports seront inclus dans un fichier ZIP contenant les codes Python produits pour chaque exercice (un par binôme). Le fichier zip nommé **nom1_nom2.zip** sera déposé sur Celene dans la zone de dépôt adéquate. Compte-tenu du barème, vous pouvez sélectionner les exercices que vous désirez faire ou ne pas faire.

Partie 0 : Prise en main de Python + OpenCV (2 pts)

Récupérez les scripts Python imageProcessing.py.txt et videoProcessing.py.txt disponibles sur Celene (avec les images Test) qui serviront de base pour les exercices.

Q1. Analysez leur contenu et ajoutez les commentaires manquants.

```
# Mon script 1 : Image_Processing OpenCV
#
import cv2
import os
import sys
import numpy as np
from matplotlib import pyplot as plt
#
print("Python version")
print (sys.version)
print (sys.version_info)
print("OPENCV Version =", cv2.__version__)
```

```
#
rep_cour = os.getcwd()
print(rep_cour)

#
img = cv2.imread('data/boats.jpg',0)

#
laplacian = cv2.Laplacian(img,cv2.CV_64F)
sobelx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=5)
sobely = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=5)
```

```
# Affichage avec OpenCV ou avec mathplotlib
cv2.namedWindow("image1", cv2.WINDOW_AUTOSIZE)
cv2.imshow("image1",img)
cv2.namedWindow("img2", cv2. WINDOW_AUTOSIZE)
cv2.imshow("image2",laplacian)
cv2.namedWindow("img3", cv2. WINDOW_AUTOSIZE)
cv2.imshow("img3",sobelx)
cv2.namedWindow("img4", cv2. WINDOW_AUTOSIZE)
cv2.imshow("img4",sobely)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
# Mon script 2 : Video_processing
#
import numpy as np
import cv2

#
def frame_processing(imgc):
    return imgc

#
cap = cv2.VideoCapture('data/terasse.mp4')

#
while (True):
    #
    ret, frame = cap.read()
    #
    if ret == True:
        #
        img = frame.copy()
        gray = cv2.cvtColor(img,
cv2.COLOR_BGR2GRAY)
        #
        gray = frame_processing(gray)
        #
        cv2.imshow('MavideoAvant', frame)
        cv2.imshow('MavideoApres', gray)
    else:
        print('video ended')
        break
    if cv2.waitKey(1000) & 0xFF == ord('q'):
        break
# When everything done, release the capture
cap.release()
cv2.destroyAllWindows()
```

Q2. Dans le script 1, on affiche les images avec OpenCV ; ci-dessous avec Matplotlib ; expliquez les différences ? Quel est le meilleur choix pour la suite ?

```
# affichage avec openCV
cv2.namedWindow("image1", cv2.WINDOW_AUTOSIZE)
cv2.imshow("image1",img)
cv2.namedWindow("image2", cv2.WINDOW_AUTOSIZE)
cv2.imshow("image2",laplacian)
cv2.namedWindow("image3", cv2.WINDOW_AUTOSIZE)
cv2.imshow("image3",sobelx)
cv2.namedWindow("image4", cv2.WINDOW_AUTOSIZE)
cv2.imshow("image4",sobely)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

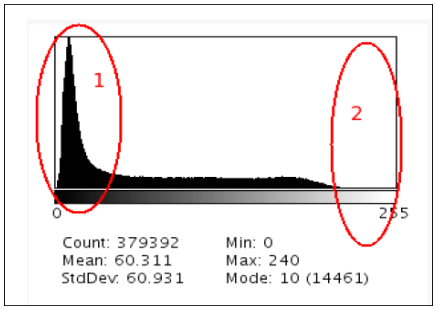
```
# Affichage avec MatPlotlib avec cmap='gray'
plt.subplot(2,2,1),plt.imshow(img,cmap = 'gray')
plt.title('Original'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,2),plt.imshow(laplacian,cmap = 'gray')
plt.title('Laplacian'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,3),plt.imshow(sobelx,cmap = 'gray')
plt.title('Sobel X'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,4),plt.imshow(sobely,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([]), plt.yticks([])
plt.show()
```

Donc, Attention notamment aux formats et fonction d'affichage et de conversion de types d'images (couleur RGB, grayscale, binary) qui divergent entre les librairies (opencv, pil,…)

Pour résoudre les exercices suivants, n'hésitez pas à parcourez les documentations en ligne (vérifier la version dont vous disposez) :
https://docs.opencv.org/3.4.2/index.html - https://opencv24-python-tutorials.readthedocs.io/en/latest/index.html

Partie 1 : Types d'images et statistiques images (2 pts)

Q1 Ouvrez l'image monalisa.png et convertissez-la en niveaux de gris. A l'aide d'openCV, générez et afficher l'histogramme de niveaux de gris, et expliquez la répartition des niveaux de gris dans l'histogramme aux points 1 et 2.
https://docs.opencv.org/3.4.2/d1/db7/tutorial_py_histogram_begins.html



Q2 Ouvrez et convertissez en niveaux de gris, les images paysage et soleil. Pour chacune d'entre-elles

- Générez l'histogramme de niveaux de gris
- Egalisez l'histogramme
- Générez l'histogramme de niveaux de gris après égalisation

Lorsque nécessaire, insérez dans votre rapport l'histogramme et l'image avant et après égalisation **et surtout expliquez**

Partie 2 : Filtrage, convolution, détection de contours (4 pts)

Q1: Ouvrez à nouveau monalisa.png (ou autre image de votre choix) et convertissez-les en niveaux de gris. Appliquez une seule fois des filtres moyenneurs de taille/rayon variable :

1. 5X5 2. 9X9 3. 15X15

Expliquez ? Que donnerait l'application d'un filtre moyenneur dont la dimension serait égale à celle de l'image ?
https://docs.opencv.org/3.4.2/d2/d96/tutorial_py_table_of_contents_imgproc.html

Q2: Appliquer d'autres traitements de filtrage passe bas (flou gaussien, masque median, …). Insérez certains résultats dans votre rapport et **commentez-les afin de démontrer que vous avez compris** le mécanisme de filtrage/convolution d'images

Q3: Effectuez les actions suivantes et reportez dans votre rapport les coefficients de filtres utilisés et les images filtrées obtenues :

- Ouvrez l'image zebre.jpg et convoluez la avec un filtre permettant de faire ressortir les traits horizontaux
- Ouvrez l'image suzan.jpg et convoluez la avec un filtre permettant de faire ressortir les traits verticaux

Expliquez les masques utilisés

Q4: Appliquer les algorithmes de détection de contours Laplacien et Canny. Insérez certains résultats dans votre rapport et commentez-les afin de démontrer que vous avez compris le mécanisme de filtrage passe-haut d'images.

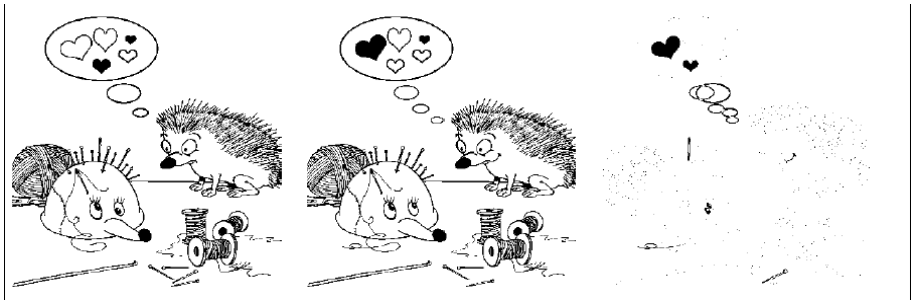
Q5: Selon vous, quels sont les points faibles de toutes ces méthodes de filtrage par convolution ?

Partie 3 : Images binaires et opérations entre images (5 pts)

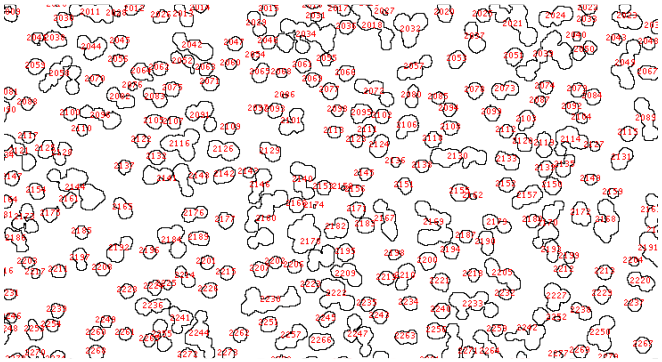
Q1: Complétez le code fourni afin d'appliquer différents algorithmes de seuillage / binarisation (seuillage fixe ou adaptatif) à différentes images fournies. Insérez certains résultats dans votre rapport et commentez-les afin de **démontrer que vous avez compris le mécanisme de binarisation**
https://docs.opencv.org/3.4.2/d2/d96/tutorial_py_table_of_contents_imgproc.html

Q2: Ouvrez les images jeu1 et jeu2.

A l'aide d'opérations arithmétiques entre ces deux images, mettez (comme sous l'image ci-dessous) en évidence les 7 différences existant entre ces deux images.
Proposez un algorithme (sans forcément l'implémenter) capable de résoudre ce problème lorsque les images à comparer sont susceptibles d'être décalées de quelques pixels l'une par rapport à l'autre (CF jeu3).



Q3: Des biologistes aimeraient compter le nombre de cellules présentes dans des images. Proposez un algorithme permettant de compter les cellules automatiquement. Vérifiez la présence de 329 cellules dans l'image cellule.png.
https://docs.opencv.org/3.4/d3/dc0/group__imgproc__shape.html#ga107a78bf7cd25dec05fb4dfc5c9e765f



Q4 : Malheureusement, une vérification faite manuellement par un biologiste indique que l'algorithme s'est trompé et que le nombre réel de cellules avoisine plutôt les 370.
Une rapide analyse de l'image vous permet de trouver l'origine de cette erreur. En effet, une mauvaise utilisation du système d'acquisition a généré deux types de bruits artificiels :
1. Des petites taches noires sont apparues un peu partout et sont interprétées comme étant des cellules (cf cas 1 de l'image ci-dessous)
2. Des cellules sont collées sur la photo alors qu'elles sont dissociées dans la réalité (cf cas 2 de l'image ci-dessous)



Trouvez la suite d'opérations morphologiques à réaliser afin que ces tâches disparaissent et que le moins de cellules possibles restent collées les unes aux autres. Un protocole "correct" devrait vous ramener vers une détection automatique de 370 cellules.
https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_morphological_ops/py_morphological_ops.html

Partie 4 : Vidéos, Détection de changement de plan et résumé automatique de vidéo (4 pts)
Q1: Complétez le code Video_processing fourni afin d'appliquer une séquence d'opérations (soustraction entre 2 frames successives, binarisation, érosion/dilatation, …) avant affichage du résultat. Expliquez l'intérêt de cette séquence (opérations entre frames successives) ?

Q2 : Changement de plan
Exploitez certaines des méthodes vues précédemment pour mettre en place un mécanisme de détection automatique de changement de plan dans les vidéos. Testez et commentez ses performances. Proposez sans le coder un mécanisme que vous jugeriez plus performant.

Q2 : Création d'Images résumés de plan

Exploiter l'ensemble des méthodes vues précédemment pour mettre en place un mécanisme de génération automatique de résumés de plans dans les vidéos. Testez et commentez ses performances. Proposez sans le coder un mécanisme que vous jugeriez plus performant.

Partie 5 :: Image Tagging (8 pts)

A vous de jouer, durant le temps de TP restant, en créant un programme associant des mot-clés décrivant le contenu (couleur, contenu, qualité …) des images et montrant vos compétences en traitement d'images. Ce programme sera décrit de manière détaillé dans votre rapport.

SI vous manquez de temps, vous pouvez néanmoins indiquer dans votre rapport ce que vous auriez mis en place avec plus de temps.

Base d'apprentissage disponible sur Celene. Ce programme devra générer des fichiers « .txt » au format similaire à ceux de la base d'apprentissage. Une image = un fichier .txt portant le même nom que l'image ; seul l'extension change ; le fichier txt est créé dans le même dossier que le fichier image.

Bon courage